

JavaScript-04

AJAX- & Fetch-exempel

28 september 2026

JavaScript – utan tjafs

Författare: Acke Strömberg



```
const object = { prop: 'Va  
const json = JSON.stringi  
fetch('/post.json', {  
  method: 'POST',  
  body: json  
});
```



För en kanin

AJAX- & Fetch-exempel

Introduktion

I filmappen för det här kapitlet finns det en undermapp som heter **uppgifter**. Där finns flera filer som används för att lära sig mer om AJAX- och Fetch-anrop. Börja med att gå in och läsa filen README.md, så kommer du med all säkerhet vara mycket klokare efter det. Kolla därefter i tur och ordning på filerna nedan.

index.html	Läs igenom noggrant
style.css	Öppna och stäng relativt snabbt
ajax.js	Den här börjar du jobba i
fetch.js	Den här blir näst på tur

Bild 1 Lista med viktiga filer.

Genomgång

När man behöver hämta data till sin webbapplikation genom ett REST API så gör man det med en asynkron funktion som heter AJAX och fetch.

Om man drar igång HTML-filen med hjälp av Live Server-tillägget i Visual Studio, så ser man en lite fjuttig webbsida med endast en rubrik och en knapp där det står Kaniner. Tanken är nu att vi ska hämta namn på kaniner från ett REST API och fylla lite kort som visar dessa på hemsidan. I filen main.js finns det en funktion som heter main().

Kommentera bort raderna med display = 'none' så kan ni se hur sidan kommer att se ut när vi har hämtat informationen. Nu låter vi dessa två rader vara med i programmet igen och då bör vi bara se rubriken och knappen igen.

Det vi vill ska hända nu är att när man trycker på knappen så gör applikationen ett HTTP-anrop till det REST API som används här. Det är

<https://jsonplaceholder.typicode.com/users> , och nu får vi alltså låtsas att det är listor av kaniner som vi hämtar, inte användare.

Event listener

Vi behöver koppla en event listener till knappen så att vi vet när vi ska hämta data. Det gör vi genom att skriva kommandot enligt bild 2.

```
knapp.addEventListener("click", hamtaKaniner)

function hamtaKaniner() {
  // INGET INNEHÅLL
}
```

Bild 2 Lägger till en eventListener till knappens click event.

När man nu klickar på knappen så kommer eventListener se till att funktionen hamtaKaniner() anropas. Vi testar att lägga in `console.log("hej")` i den funktionen. Om vi öppnar developer tools i webbläsaren med F12 så bör vi se hej i konsolen när vi klickar på knappen.

Anonyma funktioner

Det är mycket vanligt att jobba med något som kallas anonymous functions (anonyma funktioner) när man använder metoden `fetch()` i JavaScript. Det är alltså frågan om funktioner som inte har ett namn och därför inte kan anropas från andra platser i applikationen förutom där programmet just nu befinner sig. På bilderna 3 och 4 visas exakt samma hämtning av kaniner, men på bild 3 används traditionella anonymous functions och på bild 4 något som kallas för **fat arrow functions** eller bara arrow functions. Det ser ut som en fet pil, men är ett modernare skrivsätt för att visa att det är en funktion.

```
fetch("https://jsonplaceholder.typicode.com/users")
  .then(function(response) {
    return response.json()
  })
  .then(function(data) {
    console.log(data)
  })
```

Bild 3 Användandet av anonymous functions..

```
fetch("https://jsonplaceholder.typicode.com/users")
  .then((response) => {
    return response.json()
  })
  .then((data) => {
    console.log(data)
  })
```

Bild 4 Användandet av fat arrow functions.

Som man kan se blir det lättare att läsa koden enligt bild 4. Eftersom det sättet att skriva är det vanligaste, så skriver vi så fortsättningsvis.

Själva hämtet...för AJAX

Vi kommer att göra ett GET-anrop till REST API:et och skrivsättet för det är enligt bild 5. Det är nu det blir lite speciellt med den asynkrona koden. Man behöver dock inte förstå allt i detalj, men däremot försöka hänga med i den här genomgången.

```
1. function hamtaKaniner() {  
2.   let xhr = new XMLHttpRequest()  
3.   xhr.open("GET", "https://jsonplaceholder.typicode.com/users", true)  
4.   xhr.onreadystatechange = function () {  
5.     if (xhr.readyState == 4 && xhr.status == 200) {  
6.       const dataArray = JSON.parse(xhr.responseText)  
7.       skapaKaniner(dataArray)  
8.     }  
9.   }  
10.  xhr.send()  
11. }
```

Bild 5 Koden för att göra ett lyckat AJAX-anrop.

Det är raderna 2-10 som är de intressanta i det här kodstycket. På rad 2 skapar vi vår instans av ett XMLHttpRequest. Det är den som vi sedan använder några gånger för att göra anropet med. Rad 3 är själva konfigurationen med open(). Det första argumentet är vilket typ av HTTP-metod vi kör med, andra är till vilken URL vi vill skicka vår request och det tredje är om vi ska göra vår request async eller inte. På rad 4 vill vi att programkörningen går till när hämtningen är klar. Skulle vi få ett readyState == 4 och en status == 200 enligt rad 5 så vill vi göra om den textsträng som vi får tillbaka till ett JSON-objekt. Det sker på rad 6. När allt det är klart skickar vi vårt JSON-objekt till funktionen skapaKaniner(). Men rad 10 är också viktig, för det är inte förrän vi faktiskt kör send() som vi skickar något anrop till vårt REST API.

Själva hämtet...för Fetch

Vi kommer att göra ett GET-anrop till REST API:et och skrivsättet är enligt bild 6.

```
1. => function hamtaKaniner() {  
2. =>   fetch("https://jsonplaceholder.typicode.com/users")  
3. =>     .then((response) => response.json())  
4. =>     .then((data) => skapaKaniner(data))  
5. => }
```

Bild 6 Koden för att göra ett lyckat Fetch-anrop.

Det är nu det blir lite speciellt med den asynkrona koden. Man behöver dock inte förstå allt i detalj, men däremot känna till vad metoden `fetch()` skickar tillbaka om svar när man använder den. Den skickar tillbaka något som kallas `promise`. Det är ett begrepp som förekommer i JavaScript när man använder asynkrona funktioner bland annat. Ett så kallat `promise` måste man ta hand om och göra någonting med annars blir JavaScript-körningen konfunderad och börjar krångla.

Om man tittar på rad 3 i funktionen `hamtaKaniner()` så börjar den raden med en punkt och sen kommer en harang. Den där punkten visar egentligen bara att den hör ihop med uttrycket på raden ovanför, den där `fetch`-metoden som alltså skickar tillbaka ett `promise`. Det skulle gå lika bra att skriva rad 2, 3 och 4 i funktionen `hamtaKaniner()` på en enda lång rad. Problemet är bara att det är lite opraktiskt och därför delar man nästan alltid upp sådana långa rader med en punkt som börjar på raden nedanför. På rad 3 är det `.then()` som tar hand om det första `promise` som `fetch()` skickar tillbaka. Det kallas här för `response` och är alltså det svar man får från `fetch()`. Man får kalla det precis vad man själv vill, det är dock vanligt att skriva `res`, `resp` eller `response`. Var gärna så tydlig som möjligt för er egen skull. Vill ni kalla det för svar så går det alldeles utmärkt. Tänk då bara på att det sista uttrycket på rad 3 då blir `svar.json()`.

Det som händer i slutet på rad 3 är alltså att man tar svaret som kommer tillbaka från `fetch`-metoden och anropar metoden `json()` på den. Det är för att kunna göra om det svaret till någonting som är läsligt. Metoden `json()` returnerar också ett `promise` som vi tar hand om på rad 4. Där har vi kallat det vi får tillbaka från `response.json()` för `data`. Men man får kalla det för precis vad man vill här också. Vill man kalla det för `kaninInformation` så går det också bra. Sista uttrycket på rad 4 blir då `console.log(kaninInformation)`.

Vad är det vi har hämtat?

Allt är bra och vi är lyckliga, men vi har inte en aning om vad det är vi har gjort. Vi behöver titta närmare på vad vi egentligen har ställt till med. Om allt är rätt och vi har lyckats göra ett fetch-anrop så bör vi kunna se ett resultat i webbläsarens developer tools. Öppna fliken Console eller Konsol så bör man se en utskrift som ser ut ungefär som på bild 7.

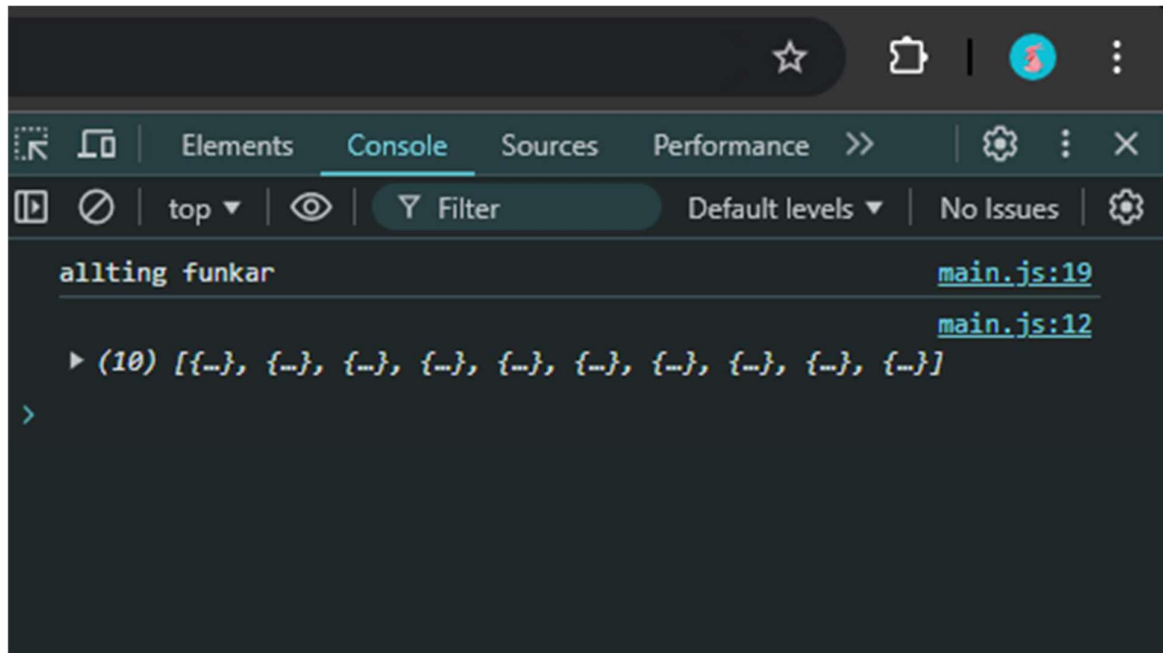


Bild 7 Utskrift av listan med objekt som vi har hämtat.

Det är den under utskriften som är den intressanta. Först en högerpil, sen en parentes med talet 10 och sen en massa måsvingar och hakparenteser. Vad är nu det här? För att begripa det här behöver man klicka på högerpilen och vips får man 10 rader som är numrerade från 0 till 9. Se närmare på bild 8.

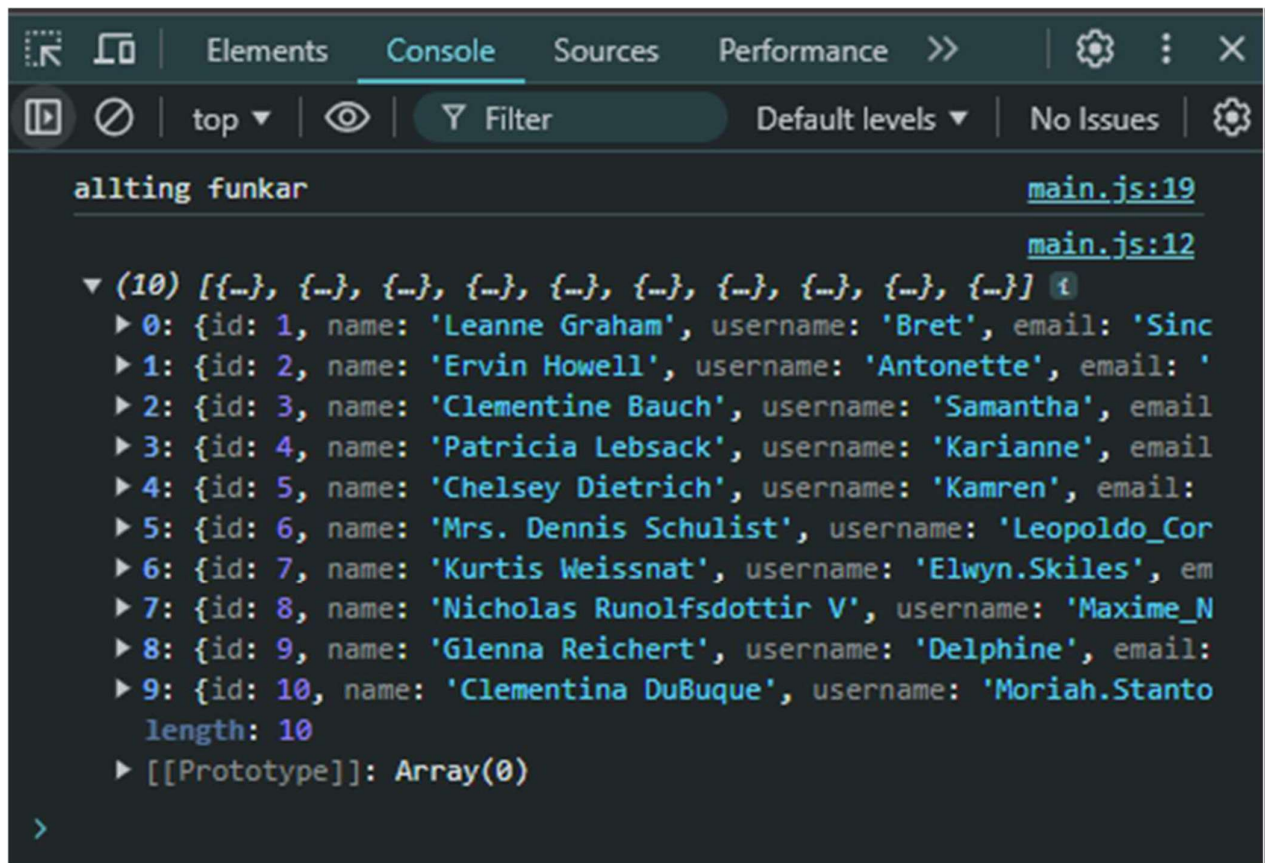


Bild 8 Varje objekt i listan, rad för rad.

Det finns ett par saker att notera här. Det är första är talet 10 i parentes på översta raden. Det säger att det finns så många objekt i listan. Att det är en lista ser vi dels genom att det står Array under alla objekt, men framför allt på den översta raden. Efter talet 10 i parentes kommer en hakparentes och flera måsvingar med tre punkter mellan sig. Om vi går vidare och klickar på den högerpil som är framför 0 i det första objektet, så får vi ytterligare lite mer information. Bild 9 är det första objektet i detalj med flera olika attribut och underattribut.

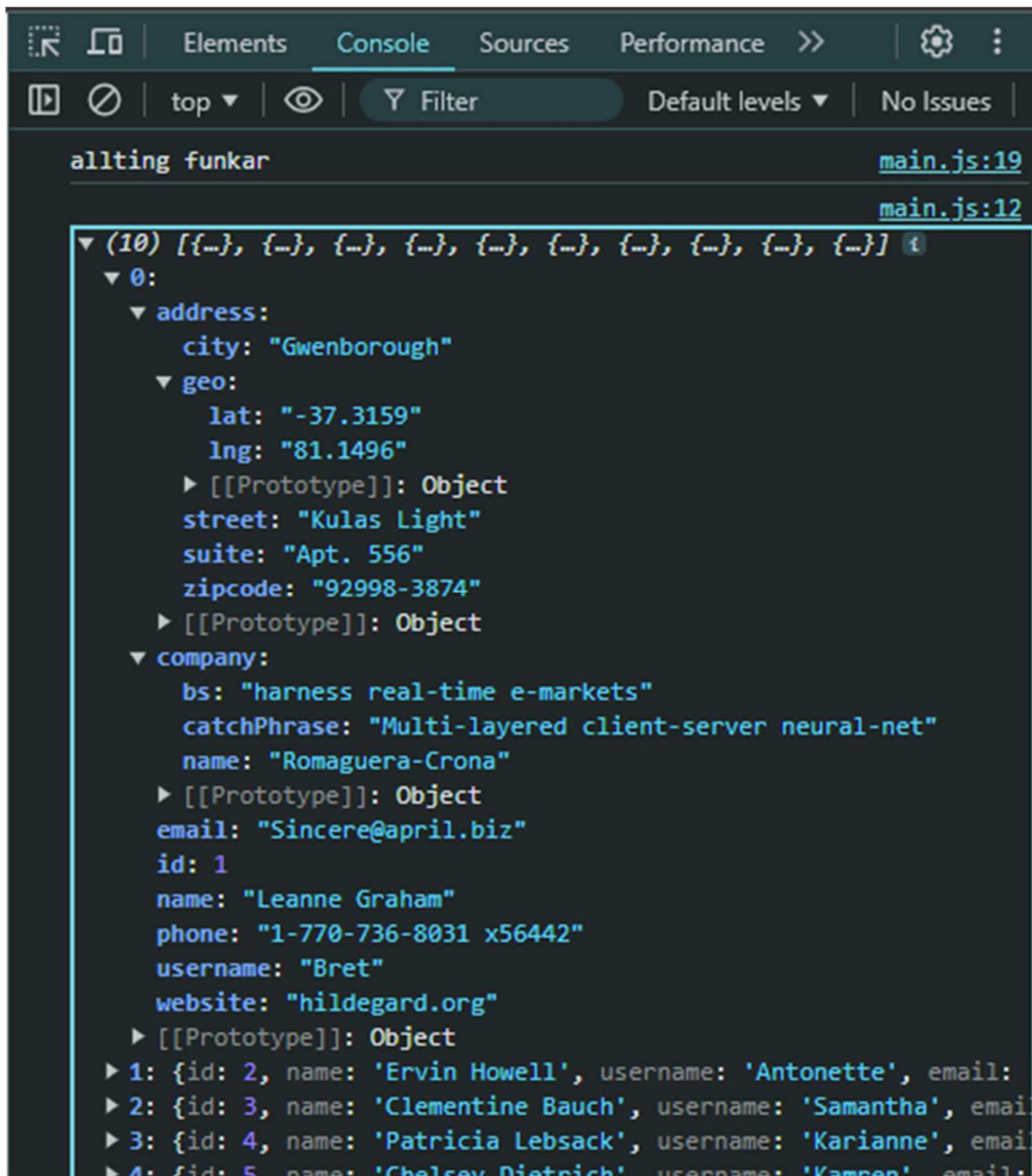


Bild 9 Första objektet i detalj, med alla attribut.

Vi har alltså fått tillbaka en lista med 10 objekt, där varje objekt har åtta huvudattribut. Det är address, company, email, id, name, phone, username och website. Sen har address och company i sin tur underattribut. Lite konstigt med kaniner som har ett telefonnummer och egen webbsida, men det är verkligen speciella kaniner det här.

Skapa kort med information

Nu är det dags gå vidare och göra något konstruktivt med all vår data från den här listan som vi har hämtat. Vi tar den data vi har hämtat och skickar den till en ny funktion - skapaKaniner(). Det första vi gör är att testa med en utskrift för att se att vi verkligen kommer till den funktionen, enligt bild 10.

```
function skapaKaniner(kaninData) {  
  console.log("--> kaninData", kaninData)  
}  
  
function hamtaKaniner() {  
  fetch("https://jsonplaceholder.typicode.com/users")  
    .then((response) => response.json())  
    .then((data) => {  
      skapaKaniner(data)  
    })  
}
```

Bild 10 En console.log i funktionen skapaKaniner.

I funktionen hamtaKaniner() ser man anropet till skapaKaniner() där vi skickar med datat som vi har fått från REST API:et. Vi kallar den parametern för kaninData och gör en utskrift av den för att kontrollera att det verkligen fungerar. Nu behöver vi i funktionen först visa rubriken och själva listan. Det kan vi göra enligt bild 11.

```
function skapaKaniner(kaninData) {  
  kaninRubrik.style.display = "block"  
  kaninLista.style.display = "block"  
}
```

Bild 11 Koden för att göra ett lyckat Fetch-anrop.

Nu borde listan med Kalle Kanin och Nisse Snubbelfot synas igen. Det andra vi vill göra är att stega igenom den lista vi har fått med oss, post för post och skapa ett nytt listelement för varje post. Där vill vi visa namnet på kaninen och någon fras från det objektet. Vi bestämmer vad när vi kommer dit. För att gå igenom en lista i JavaScript så är det bra att använda någon av de redan färdigskrivna funktionerna. Här är forEach() ett utmärkt val. Då skriver man enligt bild 12.

```
kaninData.forEach((kanin) => {  
  console.log(kanin)  
})
```

Bild 12 En console.log i funktionen skapaKaniner.

Det här är precis som en vanlig for-loop, men det är ett skrivsätt som vi föredrar. Inne i själva funktionen kommer varje listelement att vara tillgängligt för oss. Vi kan kalla det vad vi vill, men här kallar vi varje post för kanin. Efter fetpilen kommer måsvingar och innanför dessa kan vi

utföra instruktioner som vi vi göra med varje kanin. Vi vill hämta kaninens namn och sen vill vi också hämta ett attribut som ligger under company som heter catchPhrase, se bild 9. Det kan man hämta genom att skriva kanin.name och kanin.company.catchPhrase. I for-loopen måste vi skapa ett li-element, ett h3-element, ett div-element och ett p-element, och dessutom förse dessa med lämpliga attribut. Det går att göra på två olika sätt. Vi går igenom båda dessa skrivsätt, därefter får du själv välja vilket som passar dig. Skrivsätt 1 är enligt bild 13 och skrivsätt 2 enligt bild 14.

```
kaninData.forEach((kanin) => {  
  const li = document.createElement("li")  
  li.setAttribute("class", "card")  
  const h3 = document.createElement("h3")  
  h3.setAttribute("class", "card-namn")  
  h3.innerText = kanin.name  
  const div = document.createElement("div")  
  div.setAttribute("class", "card-beskrivning")  
  const p = document.createElement("p")  
  p.innerText = kanin.company.catchPhrase  
  div.appendChild(p)  
  li.appendChild(h3)  
  li.appendChild(div)  
  kaninLista.appendChild(li)  
})
```

Bild 13 Lägga till listelement enligt skrivsätt 1.

Enligt bild 13 måste man först skapa ett li-element och lägga till ett class-attribut, därefter skapa ett h3-element, lägga till ett class-attribut och innerText. Sedan går man vidare med ett div- och p-element och lägger till det som behövs. Det sista steget är att man får gå inifrån och ut genom att först appendChild av p- till div-elementet, h3- och div-elementet till li-elementet och slutligen lägga till li-elementet till kaninLista. Det var lite jobb men det går kanske att hänga med på arbetsgången?

1. Skapa respektive element
2. Lägg till attribut och text
3. Koppla varje element till sin förälder

```
kaninData.forEach((kanin) => {
  kaninLista.innerHTML += `
<li class="card">
  <h3 class="card-namn">${kanin.name}</h3>
  <div class="card-beskrivning">
    <p>${kanin.company.catchPhrase}</p>
  </div>
</li>`
})
```

Bild 14 Lägga till listelement enligt skrivsätt 2.

Skrivsätt 2 är beskrivet i bild 14, ovan och är betydligt smidigare. För varje post i listan så lägger vi till nytt innehåll till html-elementet kaninLista, genom frasen kaninLista.innerHTML. Vi vill behålla de list-element vi redan har lagt till och måste därför skriva += . Bild 15 förklarar skrivsättet.

```
// Skrivsättet
kaninLista.innerHTML += '<li class="card">\'

// Motsvarar
kaninLista.innerHTML = kaninLista.innerHTML + '<li class="card">\'
```

Bild 15 Förklaring av skrivsättet +=.

Efter += kommer något som kallas för backticks, alltså en bakåt apostrof. Det tecknet får man om man håller ner shift och knappen till vänster om Backspace. Det är också två knappar till höger om siffran 0 på tangentbordet. Lite knepigt, men du ska alltså hålla ner shift och apostrof-knappen, då händer det ingenting. Men om du nu släpper dessa knappar och trycker på mellanslag så kommer det en backtick.

Shift + apostrof => släpp dessa och tryck på mellanslag.

Bild 16 Hur man skapar en backtick.

Nu kan vi skriva något som kallas för template string inom dessa backticks. Översta raden bör alltså se ut som i bild 17 innan vi har lagt dit något innehåll.

```
kaninData.forEach((kanin) => {
  kaninLista.innerHTML += ``
})
```

Bild 17 For-loopen innan vi har lagt till innehåll, alltså bara += och backticks.

Nu kan vi gå till html-dokumentet och kopiera ett helt list-element med klasser, text och hela hootabatjooet, enligt bild 18.

```
<li class="card">
  <h3 class="card-namn">Nisse Snubbelfot</h3>
  <div class="card-beskrivning">
    <p>Nisse är glad men något klumpig.</p>
  </div>
</li>
```

Bild 18 Hela hootabatjooet som ska in mellan backticks.

Allting ska nu klistras in mellan de båda backtick-apostroferna. Men vi vill ju inte att varje post heter Nisse Snubbelfot och har hans beskrivning, utan vi vill ju hämta den från varje kanin i listan. Därför byter vi ut namnet till variabeln kanin.name. Om man nu använder det som kallas för template strings så kan man skriva in variabeln med `${kanin.name}`. Då kommer hela funktionen skapaKaniner att se ut som på bild 19.

```
function skapaKaniner(kaninData) {
  kaninRubrik.style.display = "block"
  kaninLista.style.display = "grid"
  kaninLista.innerHTML = ""
  kaninData.forEach((kanin) => {
    kaninLista.innerHTML += `
<li class="card">
  <h3 class="card-namn">${kanin.name}</h3>
  <div class="card-beskrivning">
    <p>${kanin.company.catchPhrase}</p>
  </div>
</li>`
  })
}
```

Bild 19 Allting inklusive hela hootabatjooet.

Observera att vi inte rensar bort Kalle Kanin och Nisse Snubbelfot. De är ju "riktiga" kaniner och måste ju absolut få vara med. Om allt är rätt så ska det hända lite saker när vi trycker på knappen kaniner nu. Öppna gärna developer tools med Console för att kolla eventuella felmeddelanden om inte allt fungerar.